

SHAP by Hand — a Worked Example (Shallow Tree)

OPIM 5512 · Data Science Using Python · Dr. Dave Wanik · **worked example**

SHAP explains **one** prediction: how much did each feature push it **above or below what we'd normally expect**? The "fair" split comes from **Shapley values** — a feature's *average marginal contribution* over every coalition it could join. Here you compute the whole thing by hand from a **printed decision tree**; you'll then recognize the exact same steps (on a real California-housing tree) in the notebook `2a_SHAP_ShallowTree_CAHousing`.

Warm-up: split a prize fairly (before we touch the model) 🏆 ⏳

(All amounts below are in USD.)

Two players. You and a friend win a **10,000** Kaggle prize. To split it *fairly*, a time machine lets you each redo the contest **alone**: you'd place 2nd (**7,500**), your friend 3rd (**5,000**), and with nobody playing it's **0**. Here is every way to form the team and what it's worth — the **prize** $v(S)$:

team S	$\{\}$	$\{\text{You}\}$	$\{\text{Friend}\}$	$\{\text{You, Friend}\}$
prize $v(S)$	0	7,500	5,000	10,000

Your fair share = your **average bump** over the ways you could join. With **2 players there are only $2! = 2$ orderings**, so each bump counts $\frac{1}{2}$:

- **You:** join nobody (**+7,500**) and join your friend, 3rd→1st (**+5,000**) → $\frac{1}{2}(7500) + \frac{1}{2}(5000) = \mathbf{6,250}$
- **Friend:** join nobody (**+5,000**) and join you, 2nd→1st (**+2,500**) → $\frac{1}{2}(5000) + \frac{1}{2}(2500) = \mathbf{3,750}$
- **Together:** $\mathbf{6,250 + 3,750 = 10,000 \checkmark}$

Three players. Add a third teammate → $2^3 = 8$ **teams** and $3! = 6$ **orderings**, so bumps now count $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$:

team	$\{\}$	$\{\text{P1}\}$	$\{\text{P2}\}$	$\{\text{P3}\}$	$\{\text{P1,P2}\}$	$\{\text{P1,P3}\}$	$\{\text{P2,P3}\}$	$\{\text{all}\}$
prize $v(S)$	0	5,000	5,000	0	7,500	7,500	5,000	10,000

Fair shares work out to **P1 = 5,000, P2 = 3,750, P3 = 1,250** → sum **10,000** ✓

 **Watch / read the source:** this Kaggle prize example is Conor O'Sullivan's ("A Data Odyssey").
 Video: [How SHAP values are calculated — a simple example](#) · Article: [From Shapley to SHAP — Understanding the Math](#).

 Where $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$ come from — the 6 line-ups

To credit **one** feature — say I — imagine the 3 features walking into a room in **every possible order** ($3! = 6$ of them). Each time, note **who's already inside when I walks in** — that waiting group is the team S :

order they arrive	inside when I arrives (S)
I, R, L	$\{\}$
I, L, R	$\{\}$
R, I, L	$\{R\}$
L, I, R	$\{L\}$
R, L, I	$\{R, L\}$
L, R, I	$\{R, L\}$

Count how often each team is the one waiting — that fraction **is** the weight:

- $S = \{\}$ → **2 of 6** → $\frac{1}{3}$ (I arrives first)
- $S = \{R\}$ → **1 of 6** → $\frac{1}{6}$
- $S = \{L\}$ → **1 of 6** → $\frac{1}{6}$
- $S = \{R, L\}$ → **2 of 6** → $\frac{1}{3}$ (I arrives last)

Empty and full each happen **2 ways** (the other two shuffle behind or in front, $2! = 2$) → $\frac{1}{3}$; each single-feature team happens **1 way** → $\frac{1}{6}$. The formula $\frac{|S|! (p-|S|-1)!}{p!}$ just counts this. **The 6 orders never change, so the weights never change** — same $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$ for every house. (*Two players* → only $2! = 2$ orders → $\frac{1}{2}, \frac{1}{2}$.)

Now the model — it's the 3-player game. The players are the **features** (Income, Rooms, Latitude) and the "prize" is the **prediction**. One twist: instead of starting from **0**, a model starts from the **baseline prediction** (here **10**), so the fair shares sum to *prediction* – *baseline* ($24 - 10 = 14$) and you **add the baseline back**:

$$\text{base } 10 + (\phi_I + \phi_R + \phi_L) = 24.$$

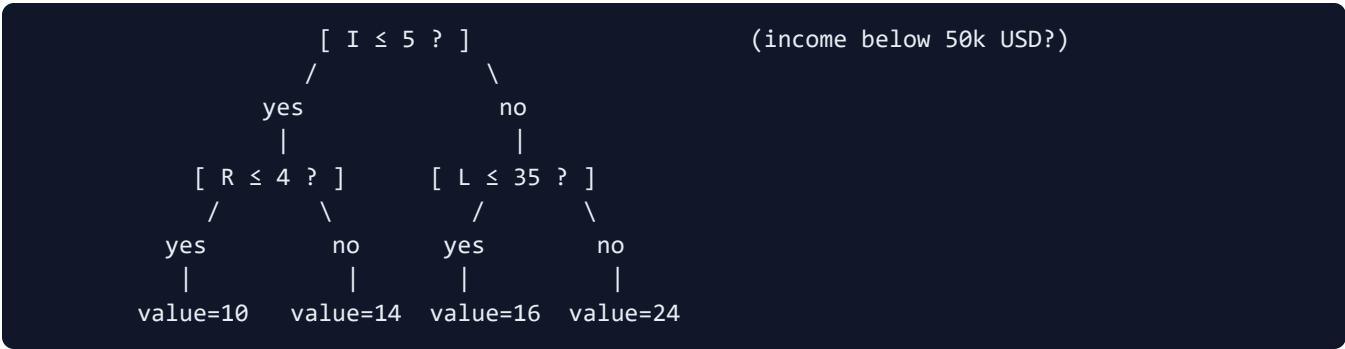
Same fair-split machinery, same $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$ weights — let's work it out on the tree.

What's what — the words to keep straight

term	plain-English meaning
coalition S	a team of features treated as "known" (e.g. $\{I, R\}$) — a <i>set</i> , not a number
the team's score $v(S)$	the model's prediction for that lineup: features in S at the home's values, the rest at baseline
the bump — $v(S \cup \{i\}) - v(S)$	how much the score jumps when feature i joins the team
the weight $w(S)$	how much each bump counts; for one feature the weights sum to 1
the SHAP value ϕ_i	feature i 's fair share of the prediction — its weighted-average bump. ← this is the answer

The model — one shallow tree

A tree scores a home (median value, in **units of 100,000 USD**) from three features: **I** = income (in units of 10,000 USD), **R** = rooms, **L** = latitude.



The value function — how we score a coalition

To score a coalition S (the features we treat as "**known**"), we set the features in S to **this home's** values and the rest to a **baseline** row, then read the tree's leaf.

Casual name for $v(S)$: call it "**the team's score**" — the price the tree predicts when only the *players* (features) in S are **on the field**, and everyone else sits at their baseline value.

feature	baseline row	this home x
I (income, 10k USD)	3	8
R (rooms)	3	6
L (latitude)	34	38

So $v(\emptyset)$ = the score with **nobody** on the field (all-baseline row) = the *starting* score, and $v(\{I, R, L\})$ = the score with **everyone** on (all-home row) = the real prediction.

Where does the baseline row come from? (This is the question everyone asks — here's the honest answer.) The **base value** is the model's **average prediction over a sample of background rows** — literally `model.predict(background).mean()`. It's *not* the prediction of one made-up "average house."

Careful — for a tree, averaging the features is NOT the same as averaging the predictions. A *linear* model lets you plug in one average row and predict once, because $f(\text{average}) = \text{average of } f$. A **tree is stepwise**, so $f(\text{average row}) \neq \text{average of } f(\text{rows})$ — you have to run the model over the *real* background rows and average the **outputs**.

`TreeExplainer` can even skip the extra data: each leaf remembers how many training houses landed in it, so it weights leaves by those counts ("**path-dependent**"); hand it a background sample and it averages `model.predict(background)` ("**interventional**").

On this worksheet we cheat: we use **one** baseline row ($I=3, R=3, L=34$) as a stand-in for the whole background so the numbers stay hand-checkable. Just know the real thing averages over many rows.

The Shapley value (coalition form), for $p = 3$ features:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} w(|S|) [v(S \cup \{i\}) - v(S)], \quad w(|S|) = \frac{|S|! (p - |S| - 1)!}{p!}.$$

👉 What this formula says, in plain English

To find a feature's SHAP value, ask: "on average, how much does this feature *bump up the score* when it joins the team?"

Here's the recipe — same four steps for **every** feature:

1. 👤 **List every team** the feature could join (all the lineups of the *other* features).
2. ➕ **Add the feature to each team** and see how much the score **jumps** — that's its **bump**: score *with it* minus score *without it*.
3. ⚖️ **Weight each bump** by w (a fixed number — see below — that just says how much that team-size counts).
4. ÷ **Add up the weighted bumps**. That total **is** the feature's SHAP value ϕ_i .

Two things that make this easier than it looks:

- 📌 **The weights are always the same.** For 3 features they're **always** $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$ — you don't recompute them, and they add up to 1, so ϕ_i is just a **weighted average of the bumps**.
- 🔄 **You do the exact same recipe for each feature** — only *which* feature is joining changes.

1 2 3 4 Where the weight comes from (it's just counting line-ups)

The scary-looking $\frac{|S|!(p - |S| - 1)!}{p!}$ is **not** something you compute per house — it's pure counting. Remember the Kaggle idea: line the features up in **every possible order** ($p!$ of them) and watch when feature i walks in. The weight for a team S is simply:

$$w(|S|) = \frac{(\text{players already inside})! \times (\text{players still to come})!}{(\text{all possible line-ups})} = \frac{|S|!(p - |S| - 1)!}{p!}$$

In plain words: "**out of all the line-ups, what fraction have exactly this team sitting there when i arrives?**" Because the *set of possible line-ups never changes*, the weights **never change** — same $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$ for every house, every dataset. Only the $v(S)$ scores move.

Every symbol in the weight (with $p = 3$)

symbol	what it is	in our tree
p	total number of players (features)	3 (Income, Rooms, Latitude)
i	the feature you're crediting right now	e.g. Income
S	a coalition — the <i>set of the OTHER features</i> already "in the room" before i joins; never contains i	one of $\{\}, \{R\}, \{L\}, \{R, L\}$
$ S $	the size of S = how many are already in	0, 1, or 2
$p - S - 1$	how many join after i	$2 - S $
$ S !$	ways to arrange those already in	
$(p - S - 1)!$	ways to arrange those still to come	
$p!$	ways to arrange everyone	$3! = 6$

Watch the sizes. When you credit one feature i , S is built from the **other** $p - 1 = 2$ features, so $|S|$ runs **0, 1, 2 — never 3** (a size-3 S would have to contain i). The "3" only appears as p and as $p! = 6$. Plugging in:

$$|S|=0 : \frac{0! 2!}{3!} = \frac{1}{3} \quad |S|=1 : \frac{1! 1!}{3!} = \frac{1}{6} \quad |S|=2 : \frac{2! 0!}{3!} = \frac{1}{3}$$

There are **two** size-1 coalitions ($\{R\}$ and $\{L\}$), each weighted $\frac{1}{6}$ — so across the four coalitions the weights read $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$ (and sum to 1).

How to read that formula, symbol by symbol

One sentence: "for every team feature i could join, measure i 's bump, weight it, and add them up."

Symbol by symbol — the **last column plugs in our problem** (with i = income):

symbol	casual meaning	our example (i = income)
N	all the features	$\{I, R, L\}$
$N \setminus \{i\}$	N without i (the $\setminus$ means "set minus")	$\{R, L\}$
$S \subseteq N \setminus \{i\}$	a team of the <i>other</i> features (the $\sum$ loops over all of them)	one of $\{\}, \{R\}, \{L\}, \{R, L\}$
$S \cup \{i\}$	union = the team S with i added in	$\{R\} \cup \{I\} = \{I, R\}$
$v(S \cup \{i\}) - v(S)$	team's score with i minus without = i 's bump	$v(\{I, R\}) - v(\{R\}) = 16 - 14 = 2$
$w(S)$	the weight (depends only on the team's size)	$\frac{1}{3}$ or $\frac{1}{6}$

Where the union $\cup$ comes in: it's just the "*i joins the team*" step. You trace the tree once with team S known ($v(S)$, so i is on baseline), then **again** with S **and** i known ($v(S \cup \{i\})$) — the difference is what i added.

The weights are constant. $w(|S|)$ depends only on the team **size** (and p) — **not** on the house. For $p = 3$ they are **always** $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$, reused for every prediction. Only the $v(S)$ numbers change from row to row.

Work it through — the whole example

We'll fill in the numbers step by step. The five results we're after:

what	value
base value $v(\emptyset)$ — the starting score	10
prediction $v(\{I, R, L\})$ — the score for this home	24
ϕ_I — income's SHAP value	+8
ϕ_R — rooms's SHAP value	+2
ϕ_L — latitude's SHAP value	+4
check: base + ϕ_I + ϕ_R + ϕ_L	$10 + 8 + 2 + 4 = \mathbf{24} \checkmark$

Now here's where each of those comes from.

Step 1 — Trace the tree for all 8 coalitions

For each coalition, set the listed features to **this home's** values ($I=8, R=6, L=38$) and leave the rest at the **baseline** ($I=3, R=3, L=34$), then follow the tree to a leaf. The all-baseline row ($3, 3, 34$) traces $I \leq 5 \rightarrow R \leq 4 \rightarrow 10$, so $v(\emptyset) = 10$.

S	$\{\}$	$\{I\}$	$\{R\}$	$\{L\}$	$\{I, R\}$	$\{I, L\}$	$\{R, L\}$	$\{I, R, L\}$
$v(S)$	10	16	14	10	16	24	14	24

Spot-checks: $v(\{I\})$: $I=8 \rightarrow I > 5 \rightarrow L=34 \leq 35 \rightarrow \mathbf{16}$. $v(\{I, L\})$: $I=8 \rightarrow I > 5 \rightarrow L=38 > 35 \rightarrow \mathbf{24}$. $v(\{R\})$: $I=3 \rightarrow I \leq 5 \rightarrow R=6 > 4 \rightarrow \mathbf{14}$. So the **base value** $v(\emptyset) = 10$ and the **prediction** $v(\{I, R, L\}) = 24$.

Step 2 — The weights (same for every feature)

With $p = 3$: $w(0) = \frac{0! 2!}{3!} = \frac{1}{3}$, $w(1) = \frac{1! 1!}{3!} = \frac{1}{6}$, $w(2) = \frac{2! 0!}{3!} = \frac{1}{3}$.

(In every table below, S ranges over the teams that do not already contain the feature — the four subsets of the other two features. Same weights each time: $\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{3}$, and they sum to 1.)

Step 3 — ϕ_I (income) — teams built from $\{R, L\}$

team S	weight w	income's bump: $v(S \cup \{I\}) - v(S)$	$w \times \text{bump}$
$\{\}$	$1/3$	$v(\{I\}) - v(\emptyset) = 16 - 10 = 6$	$6/3 = 2$
$\{R\}$	$1/6$	$v(\{I, R\}) - v(\{R\}) = 16 - 14 = 2$	$2/6 = 1/3$
$\{L\}$	$1/6$	$v(\{I, L\}) - v(\{L\}) = 24 - 10 = 14$	$14/6 = 7/3$
$\{R, L\}$	$1/3$	$v(\{I, R, L\}) - v(\{R, L\}) = 24 - 14 = 10$	$10/3$

$$\phi_I = 2 + \frac{1}{3} + \frac{7}{3} + \frac{10}{3} = \frac{6 + 1 + 7 + 10}{3} = \frac{24}{3} = \boxed{8}$$

Step 4 — ϕ_R (rooms) — teams built from $\{I, L\}$

team S	weight w	rooms's bump: $v(S \cup \{R\}) - v(S)$	$w \times$ bump
$\{\}$	$1/3$	$v(\{R\}) - v(\emptyset) = 14 - 10 = 4$	$4/3$
$\{I\}$	$1/6$	$v(\{I, R\}) - v(\{I\}) = 16 - 16 = 0$	0
$\{L\}$	$1/6$	$v(\{R, L\}) - v(\{L\}) = 14 - 10 = 4$	$4/6 = 2/3$
$\{I, L\}$	$1/3$	$v(\{I, R, L\}) - v(\{I, L\}) = 24 - 24 = 0$	0

$$\phi_R = \frac{4}{3} + 0 + \frac{2}{3} + 0 = \frac{4 + 0 + 2 + 0}{3} = \frac{6}{3} = \boxed{2}$$

Step 5 — ϕ_L (latitude) — teams built from $\{I, R\}$

team S	weight w	latitude's bump: $v(S \cup \{L\}) - v(S)$	$w \times$ bump
$\{\}$	$1/3$	$v(\{L\}) - v(\emptyset) = 10 - 10 = 0$	0
$\{I\}$	$1/6$	$v(\{I, L\}) - v(\{I\}) = 24 - 16 = 8$	$8/6 = 4/3$
$\{R\}$	$1/6$	$v(\{R, L\}) - v(\{R\}) = 14 - 14 = 0$	0
$\{I, R\}$	$1/3$	$v(\{I, R, L\}) - v(\{I, R\}) = 24 - 16 = 8$	$8/3$

$$\phi_L = 0 + \frac{4}{3} + 0 + \frac{8}{3} = \frac{0 + 4 + 0 + 8}{3} = \frac{12}{3} = \boxed{4}$$

Step 6 — Efficiency (the check)

$10 + (8 + 2 + 4) = 10 + 14 = \mathbf{24}$ = the prediction. ✓ Every dollar of the gap between the starting score (10) and this home's score (24) is handed out to exactly one feature.

Step 7 — Interactions (read from the tree)

- $v(\{I, R\}) = v(\{I\}) = 16$. Once income is high we go down the **right** branch, where the tree splits on **latitude, never rooms** — so knowing R adds nothing on top of I. Rooms only matters on the **low-income** (left) branch. That conditional relevance is a genuine **interaction**, and it's why R's fair credit ($\phi_R = 2$) is smaller than its solo effect might suggest.
- $v(\{L\}) = v(\emptyset) = 10$. With income at baseline (low) we take the **left** branch, which splits on rooms, **never latitude** — so knowing L alone changes nothing. L earns its credit ($\phi_L = 4$) entirely from coalitions where income is **high** (the right branch).

The takeaway: the base value is where you start, each feature's SHAP value is its fair *average* contribution across coalitions, and they **add up exactly** to the prediction ($10 + 8 + 2 + 4 = 24$). In the notebook you'll watch a real shallow tree produce these $v(S)$ numbers over a background sample, and the `shap` library reproduce these ϕ values.

Appendix — how the "background" *really* works (with 10 rows)

Up front we **cheated**: one baseline row (3, 3, 34), so every $v(S)$ was a single trace of the tree. In the real world you have a **background of many rows**, and each $v(S)$ is an **average over them**. Here it is with a small background of **10 rows** so you can still do it by hand.

The background — 10 rows, each traced to a leaf:

#	I	R	L	tree prediction
1	2	3	30	10
2	4	2	40	10
3	3	6	33	14
4	5	7	45	14
5	7	5	30	16
6	9	3	34	16
7	6	8	20	16
8	10	2	35	16
9	8	6	40	24
10	12	4	38	24

$v(\emptyset)$ — **nobody known**. All three features come from the background, so it's just the **average prediction**:

$$v(\emptyset) = \frac{1}{10}(10 + 10 + 14 + 14 + 16 + 16 + 16 + 16 + 24 + 24) = \frac{160}{10} = \mathbf{16}.$$

(Notice: the 1-row cheat gave 10; a real 10-row background gives 16. Different background $\rightarrow$ different base value.)

$v(\{I\})$ — **only income known**, at this home's value $I=8$. Pin $I=8$, take **R and L from each background row**, predict, and average. Because $I=8 > 5$ the tree goes right and only checks L (rooms are ignored *for this home*):

#	I (home)	R (bg)	L (bg)	prediction
1	8	3	30	16
2	8	2	40	24
3	8	6	33	16
4	8	7	45	24
5	8	5	30	16
6	8	3	34	16
7	8	8	20	16
8	8	2	35	16
9	8	6	40	24
10	8	4	38	24

$$v(\{I\}) = \frac{1}{10}(16 + 24 + 16 + 24 + 16 + 16 + 16 + 16 + 24 + 24) = \frac{192}{10} = \mathbf{19.2}.$$

So income's first **bump** is $v(\{I\}) - v(\emptyset) = 19.2 - 16 = +\mathbf{3.2}$ — and that single bump already took **10 model runs, averaged**.

The point. Every $v(S)$ except the all-known one $v(\{I, R, L\})$ (a single prediction) is an **average over the background**: keep the *known* features at the home's values, draw the *unknown* ones from each background row (R and L **together**, from the same row, to keep them realistic), predict, average. With a real background of ~ 100 rows you'd run the model **100 times per coalition** — which is exactly why the `shap` library has shortcuts. Our single baseline row on the front page just shrinks that background down to **one row** so the whole thing fits on a page.